

Sentinel AI Security Audit Report

Date: October 26, 2023
Prepared By: Sentinel AI Security Engineering Division
Subject: Automated Binary Analysis & Vulnerability Assessment
Status: CRITICAL FINDINGS IDENTIFIED

1. Executive Summary

Following a deep-learning assisted audit of the provided binary components, the Sentinel AI platform has identified a critical security flaw within the authentication subsystem. Our Graph Neural Network (GATv2) model flagged the ``vulnerable_login`` function with a **95.74% confidence rating** for memory safety violations.

Verdict: UNSAFE
Overall Risk Level: CRITICAL
Total Findings: 1 High-Confidence Vulnerability
Primary Threat: Stack-based Buffer Overflow (CWE-119 / CWE-787) leading to potential Remote Code Execution (RCE).

2. Vulnerability Analysis

Finding 1: Stack-based Buffer Overflow in ``vulnerable_login``

Attribute	Details
Function Name	``vulnerable_login``
CWE ID	[CWE-119](https://cwe.mitre.org/data/definitions/119.html),
[CWE-787](https://cwe.mitre.org/data/definitions/787.html)	
CERT C Rule	[ARR30-C](https://wiki.sei.cmu.edu/confluence/display/c/ARR30-C.+Do+not+form+or+use+out-of-bounds+pointers+or+array+subscripts)
Confidence Score	95.74%

Technical Breakdown & Exploit Vector

The disassembly reveals a classic stack-based buffer overflow vulnerability. Analysis of the Control Flow Graph (CFG) and stack frame allocation shows:

- Insufficient Buffer Allocation:** The function prologue ``SUB RSP, 0x30`` allocates 48 bytes for the stack frame.
- Unbounded Memory Copy:** In **Basic Block 0**, the instruction ``LEA RAX, [RBP - 0x10]`` calculates the address of a local buffer. This buffer is located only **16 bytes** away from the saved Base Pointer (RBP).
- The Vulnerable Call:** The instruction ``CALL 0x00003d68`` (likely a wrapper for ``strcpy`` or ``gets``) takes

user-controlled input from `RCX` and writes it into the 16-byte destination buffer.

4. **The Exploit:** Because there is no length validation (e.g., `count` or `n` parameter) before the call, an attacker providing an input string longer than 15 characters will:

- Overwrite the saved RBP.
- Overwrite the **Return Address** (located at `RBP + 0x08`).
- Gain control of the Instruction Pointer (RIP) upon the `RET` instruction in Basic Block 1.

SEI CERT C Violation

ARR30-C: The code fails to validate that the source pointer offsets remain within the bounds of the destination array. By using a fixed-size stack buffer without checking the length of the input pointer `[RBP + 0x10]`, the program permits out-of-bounds writes.

3. Remediation & Secure Code Fix

To mitigate this vulnerability, the developer must replace unsafe memory copy operations with boundary-checked alternatives.

Vulnerable Implementation (Conceptual)

```
void vulnerable_login(char *user_input) {
    char buffer[16];
    // DANGEROUS: No bounds checking
    strcpy(buffer, user_input);
    if (strcmp(buffer, "SECRET_PASS") == 0) {
        // ...
    }
}
```

Secure Remediation (C11 Standard)

The following fix utilizes `strncpy` to limit the copy operation to the size of the destination buffer and ensures null-termination.

```
#include <string.h>

#define MAX_BUF 16

void secure_login(const char *user_input) {
    if (user_input == NULL) return;

    char buffer[MAX_BUF];

    // SECURE: Limit copy to buffer size - 1
    strncpy(buffer, user_input, MAX_BUF - 1);

    // Ensure null termination
    buffer[MAX_BUF - 1] = '\0';

    if (strncmp(buffer, "SECRET_PASS", MAX_BUF) == 0) {
        // Proceed securely
    }
}
```

4. General Mitigations & Best Practices

To harden the application against similar memory safety exploits, Sentinel AI recommends the following systemic defenses:

Compiler-Level Protections

- **Stack Canaries:** Enable `-fstack-protector-all` in GCC/Clang to detect stack smashing before a function returns.
- **ASLR (Address Space Layout Randomization):** Compile with `-fPIE -pie` to randomize the memory locations of the stack, heap, and libraries.
- **NX/DEP (No-Execute):** Ensure the stack is marked as non-executable to prevent the execution of injected shellcode.
- **FORTIFY_SOURCE:** Compile with `-D_FORTIFY_SOURCE=2` to enable run-time checks for various string and memory manipulation functions.

Security Testing Integration

- **Static Analysis (SAST):** Integrate tools like `cppcheck` or `Clang Static Analyzer` into the CI/CD pipeline.
- **Fuzzing:** Utilize `AFL++` or `libFuzzer` to provide randomized, malformed inputs to the `vulnerable_login` function to identify edge-case crashes.
- **Dynamic Analysis (DAST):** Run binaries under `Valgrind` or `AddressSanitizer (ASan)` during the QA phase to catch out-of-bounds accesses in real-time.

End of Report

Sentinel AI: Securing the Future of Code.